

TRƯỜNG ĐẠI HỌC PHENIKAA
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO MÔN HỌC
MÔN HỌC: KỸ THUẬT PHẦN MỀM

Đề tài:

**HỆ THỐNG CHIA SẺ TÀI LIỆU BẢO MẬT DỰA TRÊN VÍ
BLOCKCHAIN VÀ PHÊ DUYỆT NHIỀU NGƯỜI**

Nhóm thực hiện : Nhóm 11

Sinh viên : Hoàng Tiến Đạt

Tô Thị Thùy Dương

Mã sinh viên : 23010864

Lớp học phần : Kỹ thuật phần mềm

Giảng viên :

Hà Nội – 2026

TRƯỜNG ĐẠI HỌC PHENIKAA
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO MÔN KỸ THUẬT PHẦN MỀM

**HỆ THỐNG CHIA SẺ TÀI LIỆU BẢO MẬT DỰA TRÊN VÍ
BLOCKCHAIN VÀ PHÊ DUYỆT NHIỀU NGƯỜI**

Tên hệ thống : WEB3demo
Nhóm thực hiện : Nhóm 11
Sinh viên : Hoàng Tiến Đạt; Tô Thị Thùy Dương
Công nghệ chính : Next.js, React, SQLite, MetaMask, Cloudflare R2
Hình thức triển khai : Local/Railway, storage local hoặc R2

TÓM TẮT

Báo cáo trình bày quá trình phân tích, thiết kế, cài đặt và kiểm thử hệ thống WEB3demo trong khuôn khổ môn Kỹ thuật phần mềm. Hệ thống giải quyết bài toán chia sẻ tài liệu nhạy cảm sau khi upload: tài liệu được mã hóa phía client, lưu trữ dưới dạng ciphertext, cấp quyền bằng token, hỗ trợ thu hồi quyền và cho phép thiết lập cơ chế phê duyệt nhiều ví đối với tài liệu quan trọng.

Về mặt kỹ thuật, hệ thống sử dụng Next.js và React cho giao diện, Next.js API Routes cho backend, SQLite cho metadata, MetaMask cho định danh người dùng, Web Crypto API cho mã hóa phía trình duyệt và Cloudflare R2 cho lưu trữ ciphertext khi triển khai online. Báo cáo tập trung vào các nội dung của Kỹ thuật phần mềm: khảo sát bài toán, phân tích yêu cầu, thiết kế use case, thiết kế kiến trúc, thiết kế cơ sở dữ liệu, thiết kế API, cài đặt module, kiểm thử và triển khai.

Kết quả đạt được là một prototype có thể demo theo luồng ba ví: A là chủ sở hữu tài liệu, B là người đồng phê duyệt và C là người nhận. C chỉ có thể mở tài liệu khi yêu cầu truy cập đã được A và B phê duyệt đủ ngưỡng. Hệ thống cũng cung cấp dashboard, protected viewer, audit trail, healthcheck và tài liệu triển khai Railway/R2.

LỜI CAM ĐOAN

Nhóm thực hiện cam đoan báo cáo này được xây dựng dựa trên quá trình phân tích và cài đặt hệ thống WEB3demo. Các nội dung tham khảo được ghi nhận trong phần tài liệu tham khảo. Mã nguồn, hình ảnh giao diện và kết quả kiểm thử được lấy từ dự án thực tế trong repository của nhóm.

Nhóm chịu trách nhiệm về tính trung thực của nội dung báo cáo và kết quả trình bày.

Nhóm thực hiện

LỜI CẢM ƠN

Nhóm xin gửi lời cảm ơn tới giảng viên phụ trách môn Kỹ thuật phần mềm đã định hướng các nội dung về quy trình phát triển phần mềm, phân tích yêu cầu, thiết kế hệ thống, kiểm thử và triển khai. Những kiến thức này là cơ sở để nhóm xây dựng WEB3demo như một hệ thống có cấu trúc thay vì một chức năng đơn lẻ.

Trong quá trình thực hiện, nhóm cũng tham khảo các tài liệu kỹ thuật về Next.js, Web Crypto API, MetaMask, SQLite, Cloudflare R2 và Railway để hoàn thiện sản phẩm demo.

Mục lục

TÓM TẮT

LỜI CAM ĐOAN

LỜI CẢM ƠN

DANH MỤC HÌNH ẢNH

DANH MỤC BẢNG

MỞ ĐẦU	1
1. Lý do chọn đề tài	1
2. Mục tiêu đề tài	1
3. Phạm vi đề tài	2
4. Phương pháp thực hiện	2
5. Bố cục báo cáo	2
CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI VÀ CƠ SỞ LÝ THUYẾT	4
1.1. Bài toán chia sẻ tài liệu bảo mật	4
1.2. Mã hóa phía client	4
1.3. Định danh bằng ví blockchain	5
1.4. Token truy cập và vòng đời quyền	5
1.5. Phê duyệt nhiều người	5
1.6. Audit trail	6
1.7. Cloud object storage	6

CHƯƠNG 2: KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU	7
2.1. Tác nhân hệ thống	7
2.2. Use case tổng quát	7
2.3. Yêu cầu chức năng	8
2.4. Yêu cầu phi chức năng	10
2.5. Ràng buộc và giả định	10
 CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	 11
3.1. Kiến trúc tổng quan	11
3.2. Thiết kế module	12
3.3. Thiết kế cơ sở dữ liệu	12
3.4. Thiết kế API	13
3.5. Thiết kế luồng upload và chia sẻ	14
3.6. Thiết kế luồng phê duyệt A+B cho C	14
3.7. Thiết kế giao diện	15
 CHƯƠNG 4: CÀI ĐẶT HỆ THỐNG	 20
4.1. Môi trường và công nghệ	20
4.2. Cài đặt xác thực ví	20
4.3. Cài đặt lưu trữ metadata	21
4.4. Cài đặt storage provider	21
4.5. Cài đặt upload và metadata file	21
4.6. Cài đặt phê duyệt ngưỡng	22
4.7. Cài đặt dashboard và protected viewer	22
4.8. Cài đặt triển khai	22
 CHƯƠNG 5: KIỂM THỬ, TRIỂN KHAI VÀ ĐÁNH GIÁ	 24
5.1. Chiến lược kiểm thử	24

5.2. Ma trận kiểm thử chức năng	24
5.3. Kết quả kiểm thử hiện tại	25
5.4. Triển khai local	26
5.5. Triển khai Railway và Cloudflare R2	26
5.6. Đánh giá hệ thống	27
5.6.1. Kết quả đạt được	27
5.6.2. Hạn chế	27
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	28
6.1. Kết luận	28
6.2. Bài học rút ra	28
6.3. Hướng phát triển	29
6.4. Luận điểm bảo vệ	29
TÀI LIỆU THAM KHẢO	30

Danh sách hình vẽ

1	Sơ đồ use case tổng quát của WEB3demo	8
2	Kiến trúc tổng quan của hệ thống	11
3	Sequence diagram luồng phê duyệt A+B cho C	15
4	Trang giới thiệu hệ thống	16
5	Upload wizard và thiết lập quyền chia sẻ	17
6	Dashboard quản lý tài liệu, kho và yêu cầu	18
7	Luồng nhận tài liệu và protected viewer	19

Danh sách bảng

1	Các tác nhân chính của hệ thống	7
2	Danh sách yêu cầu chức năng	8
3	Yêu cầu phi chức năng	10
4	Các module chính trong hệ thống	12
5	Các bảng dữ liệu chính	12
6	Nhóm API chính	13
7	Môi trường cài đặt	20
8	Chiến lược kiểm thử	24
9	Một số test case chính	24

MỞ ĐẦU

1. Lý do chọn đề tài

Trong môi trường học thuật và nghiên cứu, việc chia sẻ tài liệu như bài báo khoa học, luận văn, báo cáo nội bộ hoặc dữ liệu thí nghiệm diễn ra thường xuyên. Tuy nhiên, các hình thức chia sẻ phổ biến như gửi file qua email, tải lên dịch vụ lưu trữ đám mây hoặc gửi link công khai thường khiến chủ sở hữu mất dần quyền kiểm soát sau khi tài liệu đã rời khỏi thiết bị cá nhân. Người nhận có thể tải bản gốc, chuyển tiếp cho người khác hoặc giữ file sau khi quyền truy cập đáng lẽ đã hết hạn.

Vấn đề đặt ra không chỉ là "làm sao gửi được file", mà là "làm sao kiểm soát vòng đời quyền truy cập của tài liệu sau khi gửi". Đây là một bài toán phù hợp với môn Kỹ thuật phần mềm vì cần kết hợp nhiều khía cạnh: phân tích yêu cầu, thiết kế tác nhân, thiết kế cơ sở dữ liệu, thiết kế giao diện, luồng nghiệp vụ, kiểm thử, triển khai và đánh giá rủi ro.

Từ nhu cầu đó, nhóm lựa chọn đề tài **WEB3demo – hệ thống chia sẻ tài liệu bảo mật dựa trên ví blockchain và phê duyệt nhiều người**. Hệ thống không hướng tới việc thay thế hoàn toàn các nền tảng lưu trữ lớn, mà tập trung chứng minh một mô hình phần mềm trong đó tài liệu được mã hóa trước khi upload, quyền truy cập được cấp bằng token, và một số tài liệu nhạy cảm cần nhiều ví cùng phê duyệt trước khi người nhận có thể mở.

2. Mục tiêu đề tài

Đề tài hướng tới các mục tiêu chính:

- Xây dựng một prototype cho phép người dùng đăng nhập bằng ví MetaMask.
- Mã hóa tài liệu phía trình duyệt trước khi upload lên storage.
- Thiết kế cơ chế cấp, kiểm tra, thu hồi token truy cập.
- Hỗ trợ protected viewer để người nhận xem tài liệu theo quyền được cấp.
- Xây dựng cơ chế phê duyệt ngưỡng: A và B cùng duyệt thì C mới nhận quyền mở.
- Cung cấp dashboard quản lý tài liệu, token, kho nhóm, yêu cầu phê duyệt và audit trail.

- Triển khai được theo hai chế độ: local để demo ổn định và Railway/R2 để demo online.
- Kiểm thử các luồng chính bằng lint, build và Playwright E2E.

3. Phạm vi đề tài

Trong phạm vi môn học, hệ thống tập trung vào các chức năng cốt lõi của chia sẻ tài liệu bảo mật:

- Định danh người dùng bằng ví, không xây dựng hệ thống mật khẩu truyền thống.
- Mã hóa file phía client và lưu ciphertext trên local storage hoặc Cloudflare R2.
- Quản lý metadata bằng SQLite.
- Phân quyền theo token, vai trò trong kho nhóm và yêu cầu phê duyệt.
- Triển khai prototype bằng Next.js, Docker và Railway.

Đề tài chưa hướng tới các yêu cầu sản xuất quy mô lớn như phân tán database, giám sát production đầy đủ, chống chụp màn hình tuyệt đối hoặc chứng minh zero-knowledge hoàn chỉnh. Những nội dung này được xem là hướng phát triển.

4. Phương pháp thực hiện

Nhóm thực hiện đề tài theo quy trình gắn với mô hình lặp:

1. Khảo sát bài toán chia sẻ tài liệu nhạy cảm và xác định các rủi ro chính.
2. Phân tích tác nhân, use case, yêu cầu chức năng và phi chức năng.
3. Thiết kế kiến trúc tổng quan, cơ sở dữ liệu, API và luồng nghiệp vụ.
4. Cài đặt từng module: xác thực ví, upload, mã hóa, token, approval, viewer, dashboard.
5. Kiểm thử bằng lint, production build, API test và E2E test.
6. Triển khai local/Railway, cấu hình Cloudflare R2 và chuẩn bị kịch bản demo.

5. Bố cục báo cáo

Báo cáo gồm sáu chương:

- Chương 1 trình bày tổng quan đề tài và cơ sở lý thuyết.

- Chương 2 phân tích yêu cầu phần mềm.
- Chương 3 trình bày phân tích và thiết kế hệ thống.
- Chương 4 mô tả quá trình cài đặt hệ thống.
- Chương 5 trình bày kiểm thử, triển khai và đánh giá.
- Chương 6 tổng kết kết quả và hướng phát triển.

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI VÀ CƠ SỞ LÝ THUYẾT

Chương này trình bày bối cảnh bài toán và các khái niệm kỹ thuật làm nền tảng cho hệ thống WEB3demo. Các nội dung gồm chia sẻ tài liệu bảo mật, mã hóa phía client, định danh bằng ví blockchain, token truy cập, phê duyệt nhiều người, audit trail và lưu trữ object storage.

1.1. Bài toán chia sẻ tài liệu bảo mật

Chia sẻ tài liệu thông thường thường dựa trên mô hình người gửi tải file lên server, sau đó người nhận tải bản gốc xuống. Mô hình này đơn giản nhưng tạo ra nhiều rủi ro đối với tài liệu nhạy cảm. Khi bản gốc được lưu trên server hoặc đã tải về thiết bị người nhận, chủ sở hữu khó kiểm soát việc sao chép, chuyển tiếp hoặc sử dụng sau thời hạn cho phép.

Trong bối cảnh học thuật, tài liệu như bài báo chưa công bố, luận văn, đề cương nghiên cứu hoặc dữ liệu thí nghiệm có thể cần chia sẻ với người hướng dẫn, đồng tác giả hoặc hội đồng duyệt. Các tài liệu này cần cơ chế kiểm soát chặt hơn so với link tải thông thường: người nhận phải đúng danh tính, quyền truy cập có thời hạn, có thể thu hồi, và một số trường hợp cần nhiều người đồng ý trước khi mở.

1.2. Mã hóa phía client

Mã hóa phía client là phương pháp trong đó dữ liệu được mã hóa ngay trên thiết bị người dùng trước khi gửi đến server. Với cách này, server chỉ nhận ciphertext, không trực tiếp nhận plaintext. Trong WEB3demo, hướng tiếp cận này giúp giảm mức độ tin cậy bắt buộc đặt vào server.

Về mặt thiết kế, mã hóa phía client có ba lợi ích chính:

- Giảm rủi ro lộ nội dung nếu storage hoặc server bị truy cập trái phép.
- Tách rõ vai trò giữa storage ciphertext và metadata phân quyền.
- Phù hợp với mục tiêu privacy-by-design trong chia sẻ tài liệu nhạy cảm.

Hệ thống sử dụng Web Crypto API của trình duyệt để thực hiện các thao tác mật

mã. Đây là API chuẩn, tránh việc tự triển khai thuật toán mật mã bằng JavaScript thủ công.

1.3. Định danh bằng ví blockchain

Thay vì dùng tài khoản mật khẩu, hệ thống sử dụng ví MetaMask để xác thực người dùng. Quy trình cơ bản gồm:

1. Server sinh một thông điệp nonce.
2. Người dùng ký thông điệp bằng ví MetaMask.
3. Server xác minh chữ ký để biết người dùng sở hữu địa chỉ ví đó.
4. Nếu hợp lệ, server tạo session bằng JWT cookie.

Cách làm này giúp hệ thống không cần lưu mật khẩu. Địa chỉ ví trở thành định danh chính trong các bảng như tài liệu, token, thành viên kho, yêu cầu phê duyệt và audit event.

1.4. Token truy cập và vòng đời quyền

Token truy cập là mã đại diện cho quyền mở một tài liệu trong một phạm vi nhất định. Trong hệ thống, token có thể gắn với file, người nhận, thời hạn, trạng thái thu hồi và mục đích sử dụng. Token giúp tách quyền truy cập khỏi bản thân file: file vẫn tồn tại ở dạng ciphertext, còn việc người dùng có được lấy ciphertext và giải mã hay không phụ thuộc vào token và chính sách.

Vòng đời token gồm:

- Được tạo khi chủ sở hữu chia sẻ tài liệu hoặc khi yêu cầu phê duyệt đủ ngưỡng.
- Được kiểm tra trước khi người nhận mở tài liệu.
- Có thể hết hạn theo thời gian.
- Có thể bị chủ sở hữu thu hồi.
- Bị vô hiệu khi tài liệu bị hủy.

1.5. Phê duyệt nhiều người

Phê duyệt nhiều người là cơ chế yêu cầu nhiều thành viên cùng xác nhận trước khi cấp quyền truy cập. Trong WEB3demo, kịch bản trọng tâm là A và B cùng duyệt thì C

mới nhận token mở tài liệu. Cơ chế này phù hợp với tài liệu quan trọng cần kiểm soát chặt, chẳng hạn tài liệu nghiên cứu chưa công bố hoặc hồ sơ nội bộ.

Về mặt phần mềm, phê duyệt nhiều người gồm các thành phần:

- Chính sách ngưỡng trong kho nhóm.
- Các share được mã hóa cho từng thành viên.
- Yêu cầu truy cập do người nhận tạo.
- Đóng góp phê duyệt của từng người duyệt.
- Token chỉ được phát hành khi số phê duyệt đạt ngưỡng.

1.6. Audit trail

Audit trail là nhật ký ghi lại các hành động quan trọng trong hệ thống, ví dụ tạo file, tạo token, thu hồi token, duyệt yêu cầu hoặc hủy tài liệu. Audit trail không ngăn chặn rủi ro trực tiếp, nhưng giúp truy vết và giải thích vòng đời truy cập.

Trong dự án, bảng `audit_events` được thiết kế với trigger ngăn cập nhật và xóa. Điều này giúp minh họa nguyên tắc nhật ký bất biến ở mức prototype.

1.7. Cloud object storage

Cloudflare R2 được dùng như object storage để lưu ciphertext khi triển khai online. Hệ thống không lưu bản gốc lên R2; mỗi object là dữ liệu đã mã hóa. Metadata như tên file, mime type, owner, token, approval và audit được lưu ở SQLite.

Cách tách này có lợi cho thiết kế hệ thống:

- Storage chỉ cần lưu object theo khóa nội dung.
- Backend có thể đổi provider storage mà không thay đổi logic nghiệp vụ chính.
- Khi chạy local, hệ thống vẫn có thể dùng local filesystem để demo nhanh.

CHƯƠNG 2: KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương này phân tích yêu cầu hệ thống theo góc nhìn Kỹ thuật phần mềm. Nội dung gồm tác nhân, use case, yêu cầu chức năng, yêu cầu phi chức năng, ràng buộc và giả định.

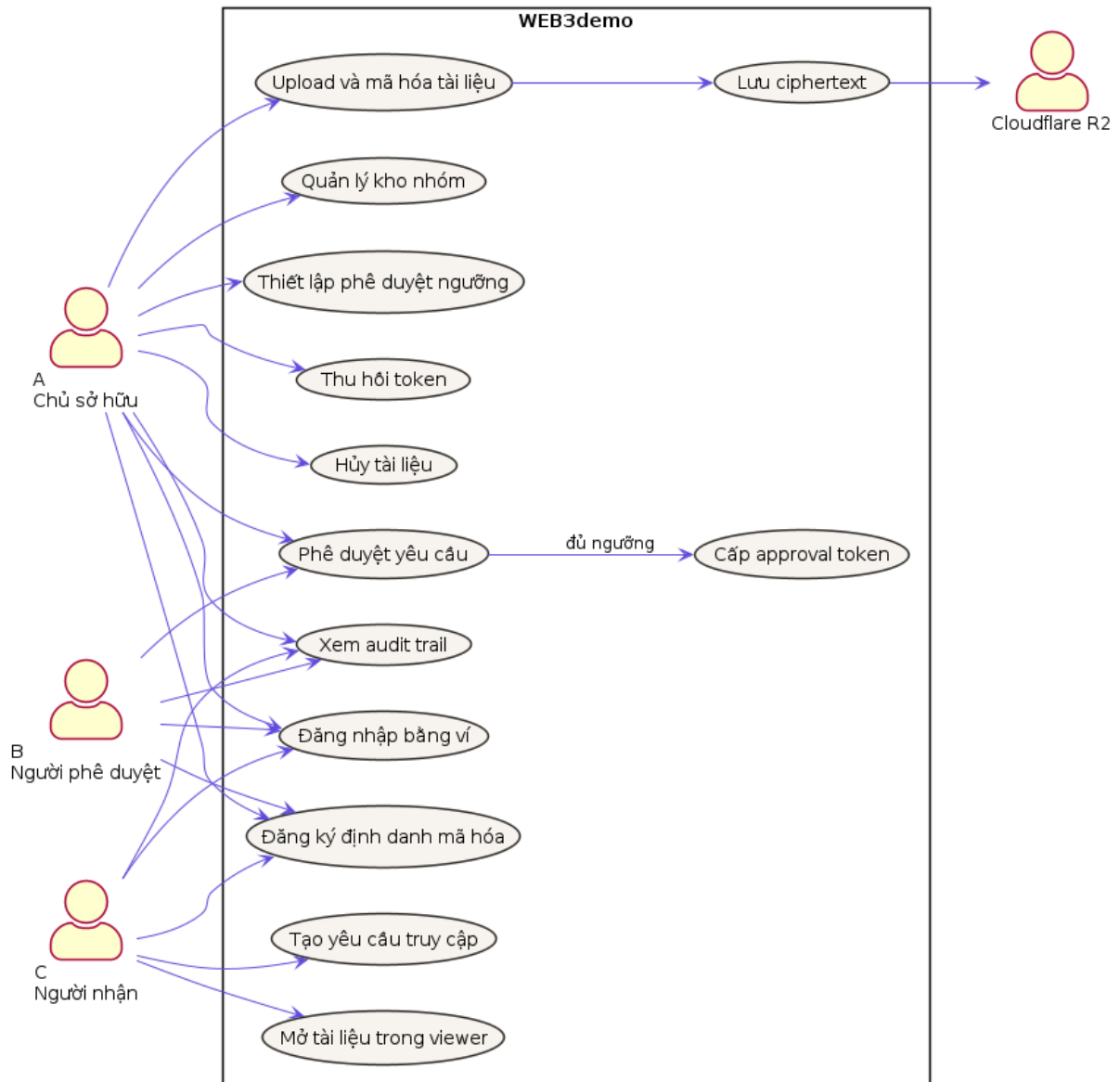
2.1. Tác nhân hệ thống

Bảng 1: Các tác nhân chính của hệ thống

Tác nhân	Mô tả
Chủ sở hữu tài liệu	Người upload, mã hóa, chia sẻ, thu hồi token và hủy tài liệu. Trong demo là ví A.
Người phê duyệt	Thành viên có quyền tham gia duyệt yêu cầu mở tài liệu nhạy cảm. Trong demo là ví B.
Người nhận	Người được cấp token hoặc tạo yêu cầu truy cập để mở tài liệu. Trong demo là ví C.
Thành viên kho	Người thuộc một kho nhóm với vai trò owner, editor hoặc viewer.
Cloudflare R2	Dịch vụ lưu trữ object, chỉ nhận ciphertext.

2.2. Use case tổng quát

Hình 1 thể hiện use case tổng quát của hệ thống. Sơ đồ được viết bằng PlantUML để dễ bảo trì và tái sinh khi thay đổi yêu cầu.



Hình 1: Sơ đồ use case tổng quát của WEB3demo

2.3. Yêu cầu chức năng

Bảng 2: Danh sách yêu cầu chức năng

Mã	Yêu cầu	Mô tả
FR-01	Đăng nhập bằng ví	Người dùng ký nonce bằng MetaMask để tạo session.

FR-02	Đăng ký định danh mã hóa	Người dùng đăng ký public key để nhận key envelope hoặc share mã hóa.
FR-03	Upload tài liệu	Chủ sở hữu chọn file, hệ thống mã hóa phía client và upload ciphertext.
FR-04	Tạo metadata file	Backend lưu cid, tên file, mime, owner, iv, access mode và thông tin khóa.
FR-05	Tạo token chia sẻ	Chủ sở hữu tạo token có thời hạn cho tài liệu.
FR-06	Kiểm tra token	Người nhận nhập token, hệ thống kiểm tra hết hạn, thu hồi và quyền ví.
FR-07	Thu hồi token	Chủ sở hữu có thể vô hiệu hóa token đã cấp.
FR-08	Hủy tài liệu	Chủ sở hữu hủy tài liệu, thu hồi token và dọn ciphertext nếu phù hợp.
FR-09	Quản lý kho nhóm	Người dùng tạo kho, thêm thành viên và gán vai trò.
FR-10	Thiết lập phê duyệt ngưỡng	Kho nhóm có thể bật chính sách K-of-N.
FR-11	Tạo yêu cầu truy cập	Người nhận tạo yêu cầu mở tài liệu cần phê duyệt.
FR-12	Phê duyệt yêu cầu	Thành viên hợp lệ gửi contribution cho yêu cầu truy cập.
FR-13	Cấp approval token	Khi đủ ngưỡng, hệ thống tạo token cho người yêu cầu.
FR-14	Protected viewer	Người nhận mở tài liệu trong viewer thay vì tải bản gốc theo mặc định.
FR-15	Audit trail	Hệ thống ghi lại các hành động quan trọng.

2.4. Yêu cầu phi chức năng

Bảng 3: Yêu cầu phi chức năng

Nhóm yêu cầu	Cách đáp ứng trong hệ thống
Bảo mật	Mã hóa phía client, JWT cookie, CSRF protection, token thu hồi, kiểm tra vai trò và audit log.
Riêng tư	Server không chủ động lưu plaintext; storage chỉ lưu ciphertext.
Khả dụng	Có thể chạy local nếu deploy online gặp sự cố; có healthcheck kiểm tra runtime.
Bảo trì	Tách module theo API, component, storage helper, db helper, audit helper và access-control helper.
Mở rộng	Storage có abstraction để dùng local hoặc Cloudflare R2; kiến trúc có thể phát triển sang IPFS/Filecoin.
Kiểm thử	Có ESLint, Next build và Playwright E2E cho các luồng quan trọng.

2.5. Ràng buộc và giả định

Các ràng buộc chính:

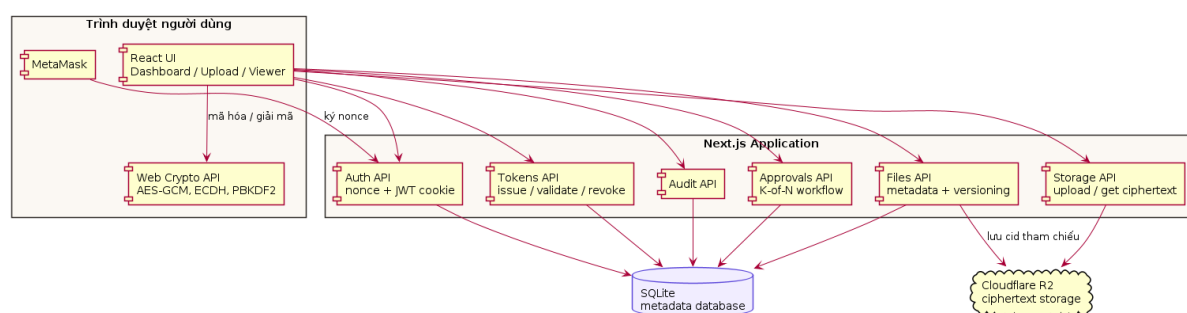
- Người dùng cần trình duyệt hỗ trợ Web Crypto API và MetaMask.
- SQLite phù hợp prototype và demo một instance; nếu scale lớn cần chuyển sang PostgreSQL hoặc database managed.
- Protected viewer không thể ngăn tuyệt đối việc người dùng chụp màn hình bằng thiết bị khác.
- R2 key phải được bảo vệ và không được commit vào repository.
- Khi deploy Railway, thư mục chứa SQLite cần được mount bằng volume.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương này trình bày thiết kế hệ thống WEB3demo ở mức kiến trúc, dữ liệu, API và luồng xử lý. Mục tiêu của chương là cho thấy hệ thống được thiết kế như một phần mềm có cấu trúc, không phải chỉ là một trang web upload file đơn giản.

3.1. Kiến trúc tổng quan

Hệ thống được chia thành ba vùng chính: trình duyệt người dùng, ứng dụng Next.js và tầng lưu trữ. Trình duyệt chịu trách nhiệm tương tác ví và mã hóa/giải mã. Backend chịu trách nhiệm xác thực, phân quyền, quản lý metadata và audit. Storage chỉ lưu ciphertext.



Hình 2: Kiến trúc tổng quan của hệ thống

Cách phân tách này có các ưu điểm:

- Logic nhạy cảm về nội dung file nằm ở client.
- Backend tập trung vào quyền truy cập và vòng đời tài liệu.
- Storage có thể thay thế giữa local filesystem và Cloudflare R2.
- Dễ kiểm thử từng tầng: UI, API, database và storage.

3.2. Thiết kế module

Bảng 4: Các module chính trong hệ thống

Module	Trách nhiệm
Auth module	Tạo nonce, xác minh chữ ký ví, tạo JWT session và đăng xuất.
Encryption identity module	Lưu public key của ví để phục vụ key envelope và threshold share.
File module	Lưu metadata file, version, access mode, vault và trạng thái hủy.
Storage module	Upload, lấy, kiểm tra và xóa ciphertext từ local hoặc R2.
Token module	Cấp, kiểm tra, liệt kê và thu hồi token truy cập.
Vault module	Quản lý kho nhóm, thành viên và vai trò.
Approval module	Quản lý yêu cầu phê duyệt, contribution và approval token.
Audit module	Ghi và truy vấn các sự kiện quan trọng.
UI module	Dashboard, upload wizard, downloader, viewer và thông báo.

3.3. Thiết kế cơ sở dữ liệu

Hệ thống sử dụng SQLite cho metadata. Dữ liệu file thực tế không lưu trong database mà lưu ở storage dưới dạng ciphertext, còn database lưu cid và thông tin quyền.

Bảng 5: Các bảng dữ liệu chính

Bảng	Vai trò
files	Lưu metadata tài liệu: owner, cid, tên file, mime, kích thước, iv, access mode, version, vault, trạng thái hủy.
tokens	Lưu token truy cập, file tương ứng, người nhận, thời hạn, trạng thái thu hồi và mục đích token.
encryption_identities	Lưu public key của ví và chữ ký xác nhận public key thuộc về ví đó.
key_envelopes	Lưu key envelope cho token đích danh.
vaults	Lưu thông tin kho nhóm.
vault_members	Lưu thành viên kho và vai trò owner/editor/viewer.

vault_threshold_policies	Lưu chính sách phê duyệt ngưỡng của kho.
threshold_file_shares	Lưu các share đã mã hóa cho từng thành viên.
approval_requests	Lưu yêu cầu truy cập của người nhận.
approval_contributions	Lưu đóng góp phê duyệt của từng người duyệt.
audit_events	Lưu nhật ký hành động quan trọng; có trigger ngăn sửa/xóa.
rate_limits	Lưu bộ đếm giới hạn request cho API nhạy cảm.

Một số quan hệ quan trọng:

- Một file có thể có nhiều token.
- Một vault có nhiều thành viên và một chính sách phê duyệt.
- Một yêu cầu phê duyệt thuộc về một file và một requester.
- Một yêu cầu có nhiều contribution từ các người duyệt khác nhau.
- Audit event gắn với actor, action, resource type và resource id.

3.4. Thiết kế API

Bảng 6: Nhóm API chính

Phương thức	Endpoint	Mục đích
POST	/api/auth/start	Sinh thông điệp nonce để ví ký.
POST	/api/auth/verify	Xác minh chữ ký ví và tạo session.
POST	/api/storage/upload	Nhận ciphertext và lưu vào provider storage.
GET	/api/storage/get	Trả ciphertext nếu token hoặc approval hợp lệ.
POST	/api/files	Tạo metadata file, token và share nếu có.
DELETE	/api/files/[id]	Chủ sở hữu hủy tài liệu.
POST	/api/tokens/issue	Cấp token mới cho file.

POST	/api/tokens/validate	Kiểm tra token trước khi mở file.
POST	/api/tokens/revoke	Thu hồi token.
GET	/api/approvals	Liệt kê yêu cầu phê duyệt liên quan đến người dùng.
POST	/api/approvals/[id]/approve	Gửi contribution phê duyệt cho yêu cầu.
GET	/api/audit	Truy vấn audit event được phép xem.
GET	/api/health	Kiểm tra database và provider storage.

3.5. Thiết kế luồng upload và chia sẻ

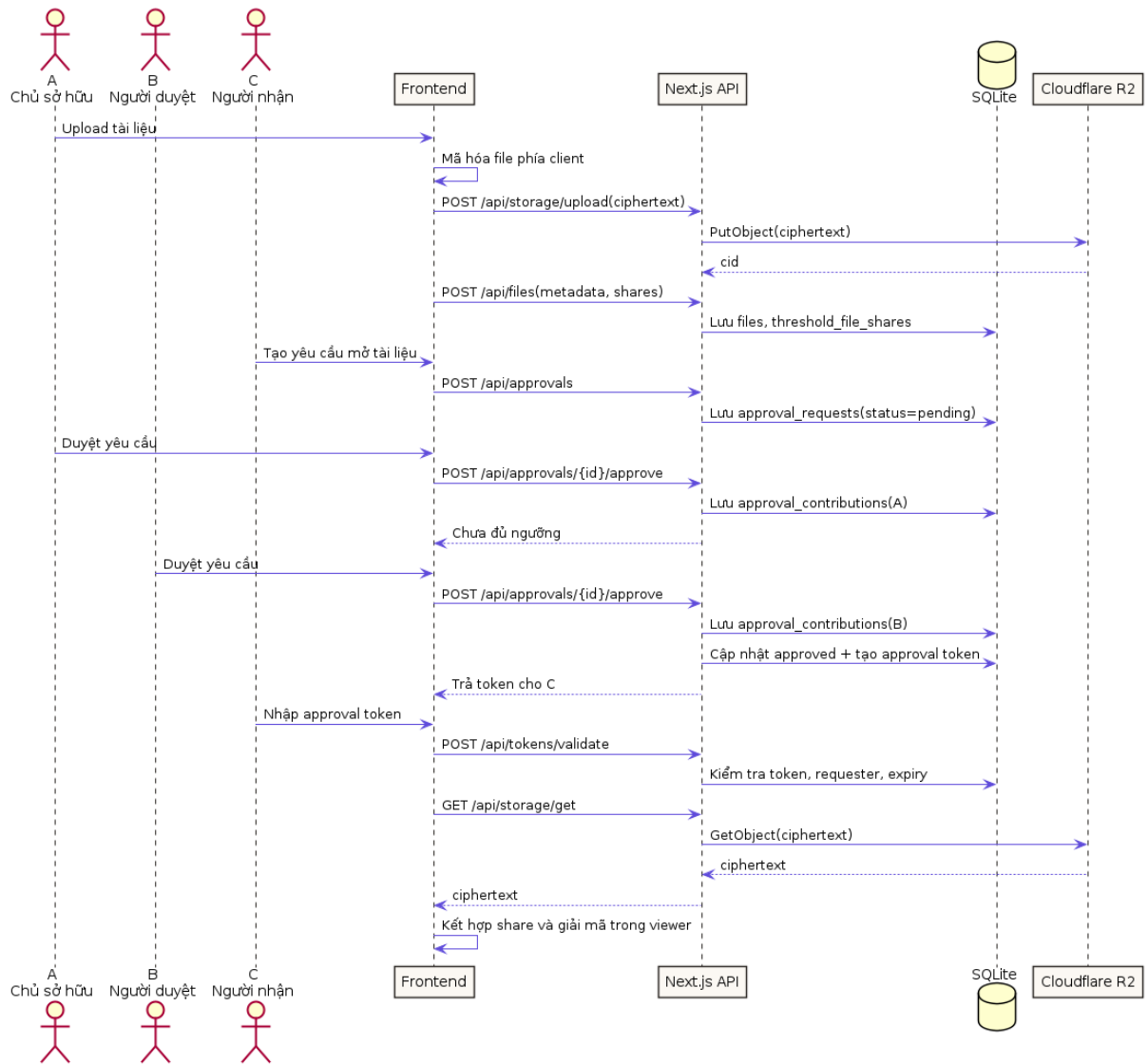
Luồng upload gồm các bước:

1. Người dùng kết nối ví và có session hợp lệ.
2. Người dùng chọn file trong upload wizard.
3. Trình duyệt tạo khóa mã hóa và mã hóa file bằng Web Crypto.
4. Client upload ciphertext lên /api/storage/upload.
5. Storage trả về cid.
6. Client gửi metadata, iv, access mode và key material hợp lệ tới /api/files.
7. Backend lưu metadata, tạo token ban đầu và ghi audit event.

Điểm quan trọng là file rõ chỉ tồn tại tại trình duyệt. Backend không nhận file gốc trong thiết kế chủ đích.

3.6. Thiết kế luồng phê duyệt A+B cho C

Luồng phê duyệt được thiết kế để minh họa kiểm soát truy cập nhiều người. Hình 3 mô tả trình tự chính.



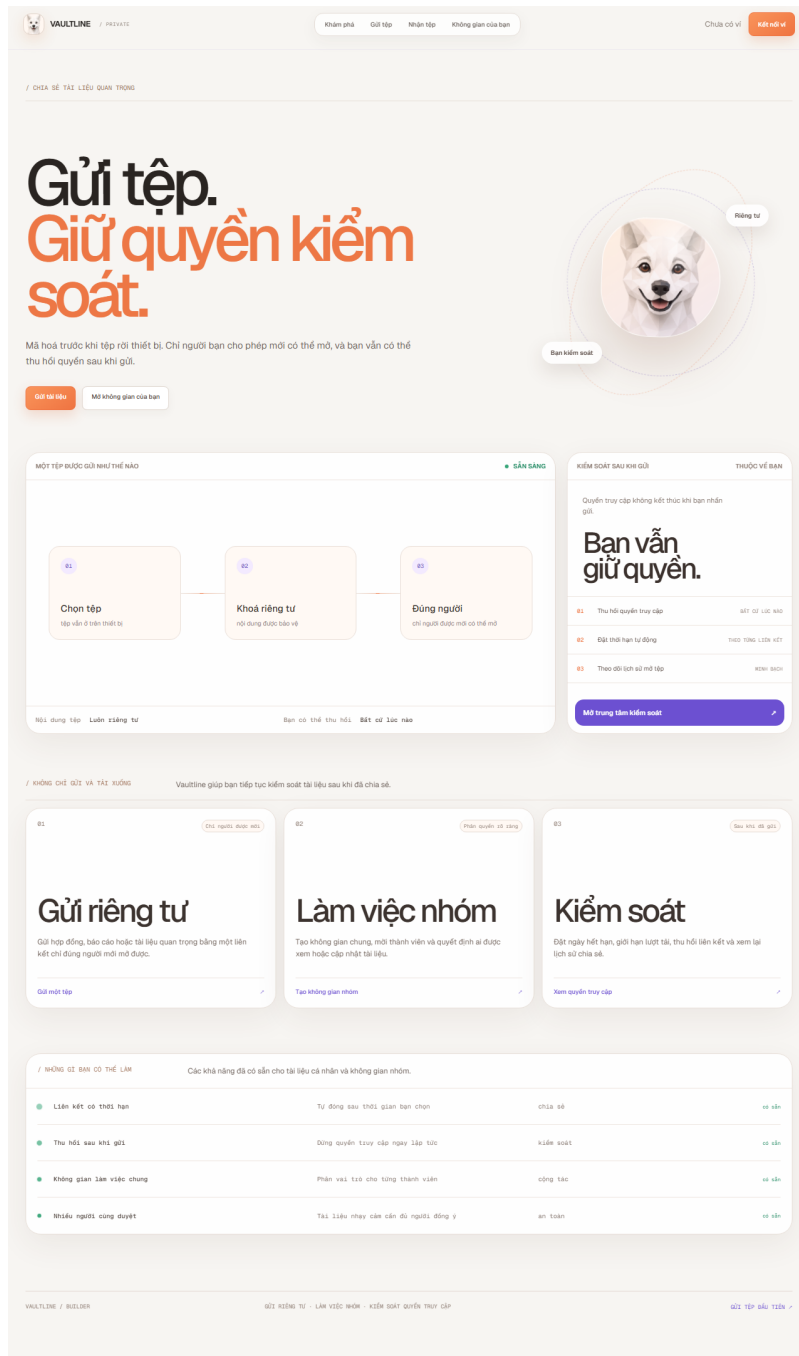
Hình 3: Sequence diagram luồng phê duyệt A+B cho C

Các quy tắc chính:

- Người yêu cầu không được tự phê duyệt yêu cầu của mình.
- Contribution của mỗi người duyệt chỉ được tính một lần.
- Token approval chỉ được tạo khi số contribution hợp lệ đạt ngưỡng.
- Token approval gắn với requester, không phải token mở tự do cho mọi ví.

3.7. Thiết kế giao diện

Giao diện gồm bốn màn hình chính: giới thiệu, upload, dashboard và viewer. Các màn hình dùng chung hệ nhận diện, chuyển động và logo của dự án.



Hình 4: Trang giới thiệu hệ thống

VAULTLINE

/ PRIVATE

Khám phá

Gửi tệp

Nhận tệp

Không gian của bạn

Chưa có ví

Kết nối ví

GỬI TẬP RIÊNG TƯ

Chọn tệp. Chọn người được xem.

Tạo một liên kết riêng tư, đặt thời hạn hoặc giới hạn lượt tải, rồi gửi đi mà vẫn giữ quyền thu hồi.

Kết nối ví để gửi tài liệu

Vì xác nhận bạn là chủ tài liệu và cho phép bạn quản lý hoặc thu hồi liên kết sau khi gửi.

Thông tin

Mã hoá và tải lên

Chia sẻ

Thông tin cơ bản

Đặt tên để bạn dễ nhận ra tài liệu này sau khi gửi.

Tiêu đề

Tài liệu được mã hoá của tôi

Mô tả

Ghi chú tùy chọn về tệp này

Liên kết còn hiệu lực trong

1 ngày

Yêu cầu mật khẩu khi mở

1 giờ

6 giờ

24 giờ

3 ngày

Tùy chọn

Tùy chọn - tăng mức bảo mật

Người nhận cần nhập đúng mật khẩu để mở tài liệu.

Cần mật khẩu hoặc ví người nhận.

Tùy chọn

Cách người nhận sử dụng tài liệu

Chỉ đọc được kiểm soát

Nếu tin trong trình xem, gần watermark và không cung cấp nút tải file.

Cho phép tải gửi mã hóa

Vẫn xem bằng Viewer; tệp tải về là gói 'vaultline', không phải bản gốc.

Phù hợp hơn cho bản thảo và bài báo khoa học cần theo dõi người đọc.

Tự hủy sau số lượt mở

0

Nhập 0 để không giới hạn. Bản mã sẽ bị xoá sau lượt mở cuối cùng được cho phép.

Chỉ cho phép một người nhận

Địa chỉ Ox người nhận, không bắt buộc

Khi thiết lập, chỉ tài khoản này có thể mở tài liệu. Mật khẩu sẽ không được sử dụng.

Quản lý phiên bản

Tạo một tệp mới

Bắt đầu một lịch sử phiên bản riêng

Mỗi phiên bản được lưu riêng để bạn có thể xem lại lịch sử thay đổi.

Lịch lưu trữ

Tệp cá nhân

Chỉ bạn quản lý tệp này

Chú kho và biến tệp viên có thể tải tệp mã hoá lên.

Mã hoá và tải lên

Kết nối ví để lưu

Thả tệp vào đây

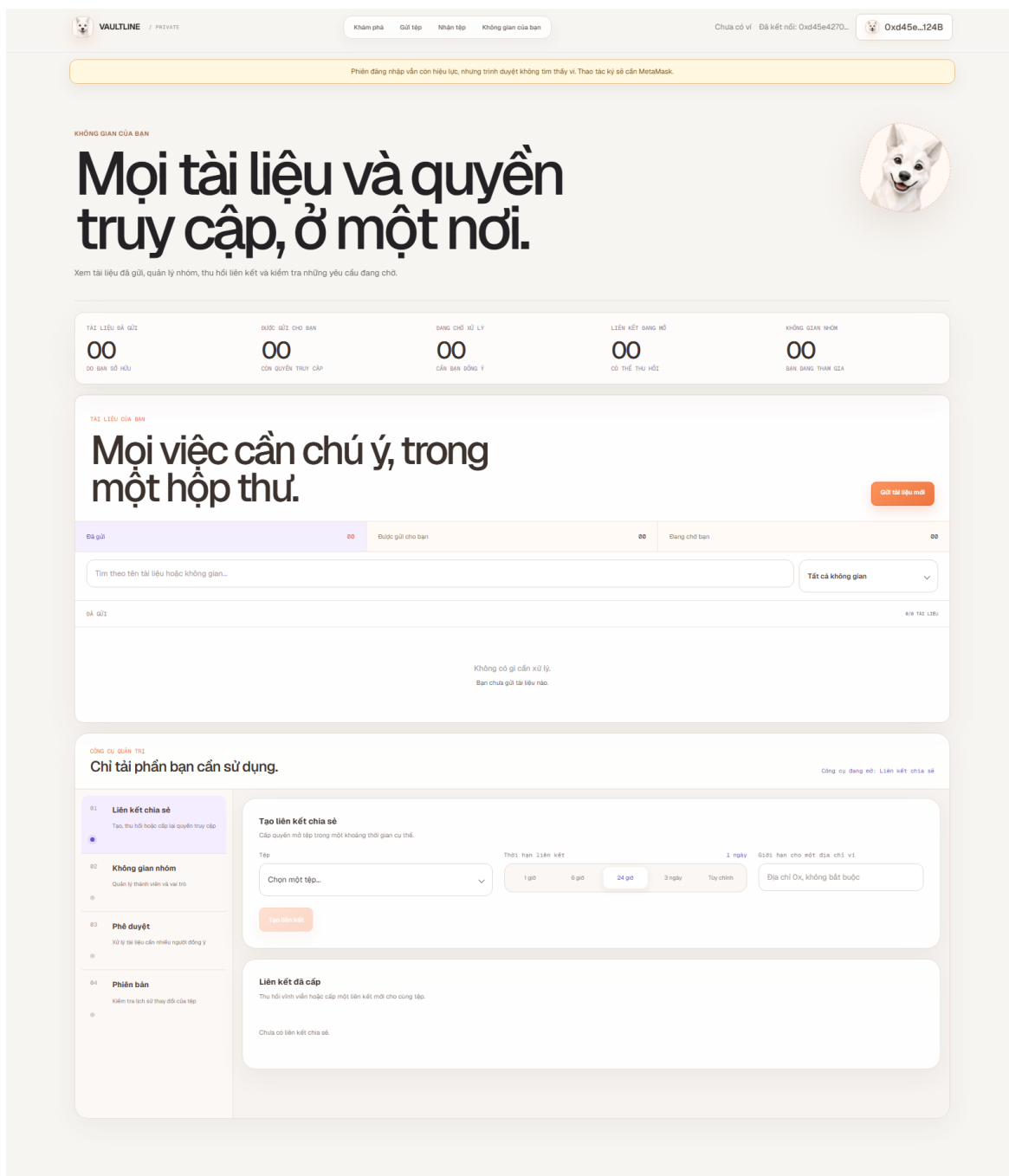
hoặc

Chọn tệp

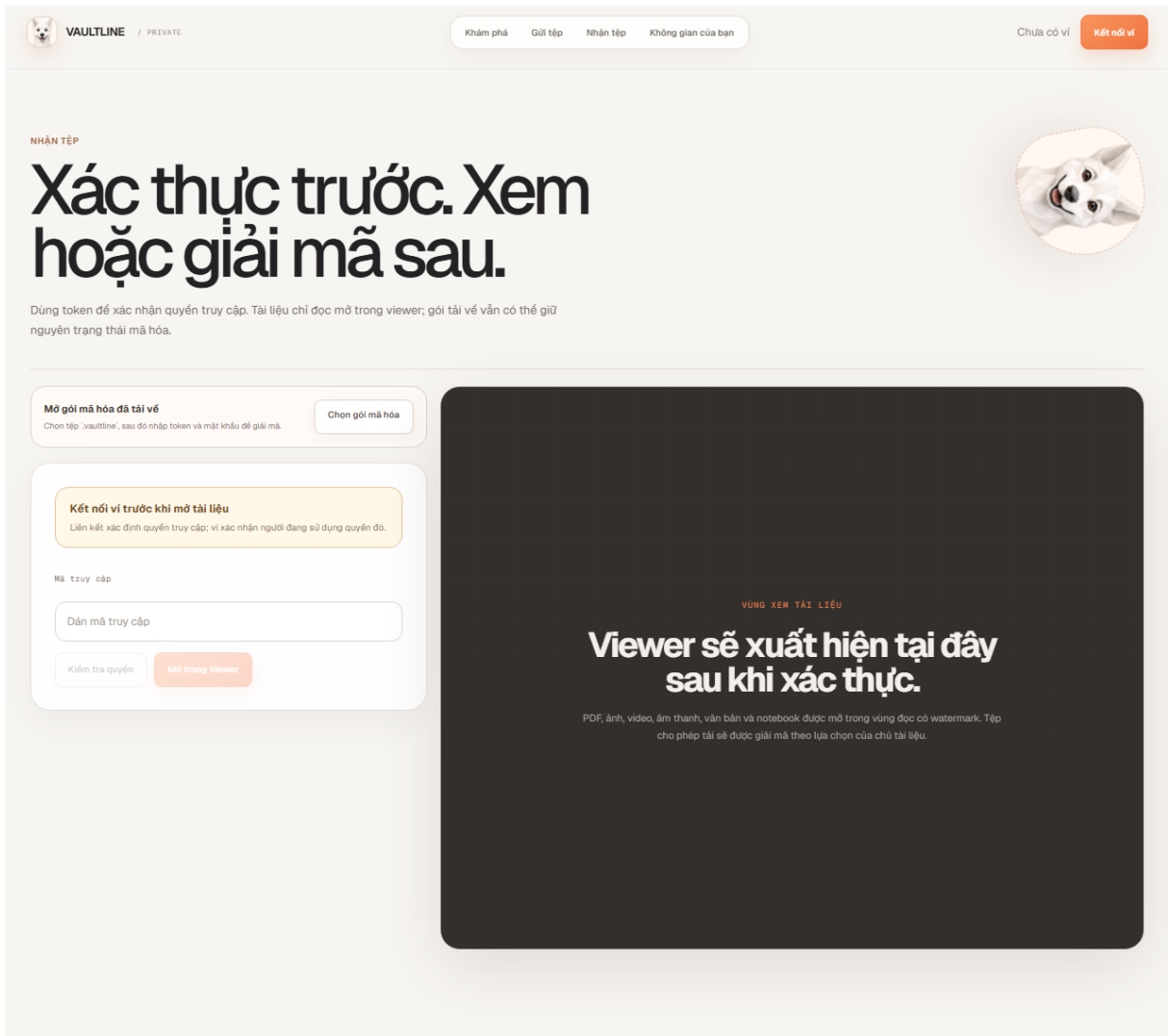
Tối đa khoảng 20MB cho bản demo

Hình 5: Upload wizard và thiết lập quyền chia sẻ

17



Hình 6: Dashboard quản lý tài liệu, kho và yêu cầu



Hình 7: Luồng nhận tài liệu và protected viewer

CHƯƠNG 4: CÀI ĐẶT HỆ THỐNG

Chương này mô tả cách các thiết kế ở chương trước được hiện thực trong mã nguồn. Nội dung tập trung vào các module quan trọng: xác thực ví, mã hóa, storage, token, approval, dashboard và triển khai.

4.1. Môi trường và công nghệ

Bảng 7: Môi trường cài đặt

Thành phần	Công nghệ
Frontend	Next.js 15, React 19, TypeScript
Styling/UI	CSS module/global CSS, component tự xây dựng
Wallet	MetaMask, Ethers.js
Mã hóa	Web Crypto API, AES-GCM, ECDH/PBKDF2 theo luồng tương ứng
Backend	Next.js API Routes chạy Node.js runtime
Database	SQLite với better-sqlite3
Storage	Local filesystem hoặc Cloudflare R2 qua AWS S3 SDK
Deploy	Dockerfile, Railway, persistent volume
Kiểm thử	ESLint, Next build, Playwright

4.2. Cài đặt xác thực ví

Xác thực ví được tách thành hai endpoint. Endpoint `/api/auth/start` sinh thông điệp cần ký. Endpoint `/api/auth/verify` nhận địa chỉ ví và chữ ký, sau đó dùng thư viện ethers để khôi phục địa chỉ ký. Nếu địa chỉ khôi phục trùng với địa chỉ gửi lên, server tạo session JWT và lưu trong cookie.

Cách cài đặt này giúp hệ thống:

- Không cần lưu mật khẩu.
- Ràng buộc hành động với địa chỉ ví.
- Dễ mở rộng sang các chính sách phân quyền theo ví.

4.3. Cài đặt lưu trữ metadata

Module `src/lib/db.ts` chịu trách nhiệm mở SQLite database và chạy migration. Đường dẫn database lấy từ biến môi trường `DB_PATH`. Khi chạy local, đường dẫn thường là `data/app.sqlite`. Khi chạy Railway, đường dẫn là `/app/data/app.sqlite` để dữ liệu nằm trong volume.

Migration được viết trực tiếp trong code để prototype dễ chạy. Khi database chưa có bảng, hệ thống tự tạo bảng. Khi phát triển thêm trường mới, code kiểm tra PRAGMA `table_info` và thêm cột còn thiếu. Cách này phù hợp với phạm vi môn học vì giảm bước cấu hình thủ công.

4.4. Cài đặt storage provider

Module `src/lib/storage.ts` cung cấp cùng một interface cho hai provider:

- local: ghi ciphertext vào thư mục `storage/`.
- r2: ghi ciphertext vào Cloudflare R2 bằng S3-compatible API.

Provider được chọn bằng biến môi trường `STORAGE_PROVIDER`. Khi dùng R2, hệ thống yêu cầu account ID, bucket, access key ID và secret access key tương ứng. Các giá trị bí mật chỉ được đặt trong biến môi trường.

Để tránh trùng object, cid được tạo bằng SHA-256 trên ciphertext. Vì cid sinh từ ciphertext, cùng một ciphertext sẽ cho cùng cid.

4.5. Cài đặt upload và metadata file

Khi client gửi metadata tới `/api/files`, backend kiểm tra:

- Người dùng đã đăng nhập hay chưa.
- CSRF token có hợp lệ không.
- Các trường bắt buộc như cid, iv.
- Kích thước file có vượt giới hạn không.
- Key material thuộc đúng một chế độ: wrapped key, recipient envelope hoặc threshold shares.

- Nếu file thuộc vault, người dùng có quyền ghi hay không.

Sau khi hợp lệ, hệ thống lưu bản ghi vào bảng files, tạo token, lưu share nếu có và ghi audit event.

4.6. Cài đặt phê duyệt ngưỡng

Phê duyệt ngưỡng được hiện thực bằng nhóm bảng threshold file, threshold share, approval request và approval contribution. Khi người nhận tạo yêu cầu, hệ thống lưu trạng thái pending. Khi người duyệt gửi approval, API kiểm tra tư cách thành viên và lưu contribution.

Khi số contribution hợp lệ đạt ngưỡng, hệ thống:

1. Cập nhật yêu cầu sang trạng thái approved.
2. Tạo token mục đích approval.
3. Gắn token với requester.
4. Cho phép requester dùng token để lấy ciphertext và contribution cần thiết.

4.7. Cài đặt dashboard và protected viewer

Dashboard tập hợp nhiều chức năng vào một workspace: tài liệu đã gửi, tài liệu nhận, kho nhóm, yêu cầu phê duyệt, token và phiên bản. Các component được tách theo từng nhóm chức năng để giảm phụ thuộc và thuận tiện bảo trì.

Protected viewer nằm trong luồng download. Viewer chỉ hiển thị nội dung sau khi token hợp lệ và client giải mã thành công. Với một số loại file như ảnh, video hoặc notebook, viewer có thể hiển thị theo kiểu tương ứng; với loại chưa hỗ trợ, hệ thống vẫn có thể cung cấp trạng thái rõ ràng cho người dùng.

4.8. Cài đặt triển khai

Repository có Dockerfile và railway.json. Dockerfile build Next.js ở chế độ standalone. Railway dùng healthcheck /api/health. SQLite được cấu hình tại /app/data/app.sqlite. Cloudflare R2 được cấu hình qua biến môi trường.

Một lỗi thực tế khi triển khai là quyền ghi vào Railway volume. Vì Railway mount

volume ở runtime, container cần đảm bảo tiến trình có quyền ghi vào /app/data. Dự án đã điều chỉnh Dockerfile để tránh lỗi SQLite không ghi được vào volume trong demo.

CHƯƠNG 5: KIỂM THỬ, TRIỂN KHAI VÀ ĐÁNH GIÁ

Chương này trình bày chiến lược kiểm thử, kết quả kiểm tra, phương án triển khai và đánh giá hệ thống sau khi cài đặt.

5.1. Chiến lược kiểm thử

Hệ thống được kiểm thử theo nhiều tầng:

Bảng 8: Chiến lược kiểm thử

Tầng kiểm thử	Công cụ	Mục tiêu
Static analysis	ESLint	Phát hiện lỗi cú pháp, lỗi React/TypeScript và quy tắc code.
Production build	Next.js build	Kiểm tra khả năng compile, type check và build production.
API integration	Playwright request	Kiểm tra auth, token, approval, vault, audit và phân quyền.
End-to-end	Playwright Chromium	Kiểm tra các luồng người dùng trên giao diện.
Runtime healthcheck	/api/health	Kiểm tra database và storage provider khi chạy thật.

5.2. Ma trận kiểm thử chức năng

Bảng 9: Một số test case chính

Mã	Kịch bản	Kỳ vọng
TC-01	Truy cập các trang public	Không có lỗi hydration React.
TC-02	Mở download khi chưa đăng nhập	Nút kiểm tra và mở tài liệu bị vô hiệu hóa.
TC-03	Gọi healthcheck	API trả về database ready và storage provider.

TC-04	Đăng ký encryption identity	Public key ký bởi đúng ví được lưu; chữ ký ví khác bị từ chối.
TC-05	Tạo token và validate token	Token hợp lệ được chấp nhận; token thu hồi/hết hạn bị từ chối.
TC-06	Phân quyền vault	Owner/editor/viewer bị giới hạn đúng vai trò.
TC-07	Phê duyệt ngưỡng	Một approval chưa đủ cấp token; đủ ngưỡng mới cấp token cho requester.
TC-08	Hủy tài liệu	Chỉ owner được hủy; token liên quan bị thu hồi.
TC-09	Audit trail	Hành động nhạy cảm sinh audit event và event không thể sửa/xóa.
TC-10	Nhãn UI	Button/link hiển thị có label để dễ dùng và dễ kiểm thử.

5.3. Kết quả kiểm thử hiện tại

Các lệnh kiểm tra thường dùng:

```
npm run lint
npm run build
npm run test:e2e
```

Trong quá trình hoàn thiện, nhóm đã chạy lint và build nhiều lần. Production build thành công cho các route chính như trang chủ, upload, download, dashboard và các API route. E2E test bao phủ các luồng như hydration, đăng nhập giả lập bằng ví, kiểm tra token, approval, vault, audit, account menu và quyền truy cập.

5.4. Triển khai local

Local là phương án demo dự phòng quan trọng. Khi chạy local, app server chạy trên máy cá nhân nhưng vẫn có thể dùng Cloudflare R2 để lưu ciphertext. Điều này giúp demo được đầy đủ logic cloud storage ngay cả khi Railway gặp sự cố.

Cấu hình local cơ bản:

```
JWT_SECRET=replace-with-a-long-random-secret
REQUIRE_CSRF=true
DB_PATH=data/app.sqlite
STORAGE_PROVIDER=r2
CLOUDFLARE_R2_ACCOUNT_ID=<account_id>
CLOUDFLARE_R2_BUCKET=<bucket_name>
CLOUDFLARE_R2_ACCESS_KEY_ID=<rotated_access_key_id>
CLOUDFLARE_R2_SECRET_ACCESS_KEY=<rotated_secret_access_key>
CLOUDFLARE_R2_KEY_PREFIX=ciphertexts
```

Chạy production local:

```
npm run build
npm run start
```

5.5. Triển khai Railway và Cloudflare R2

Triển khai online gồm hai phần: Railway chạy ứng dụng và Cloudflare R2 lưu ciphertext. SQLite được lưu trong Railway volume mount tại /app/data. Các biến môi trường quan trọng gồm JWT secret, đường dẫn database, storage provider và các R2 access key.

Healthcheck sau khi deploy:

```
https://<domain>.up.railway.app/api/health
```

Kết quả mong đợi:

```
{
  "ok": true,
  "database": "ready",
  "storageProvider": "r2"
```

}

5.6. Đánh giá hệ thống

5.6.1. Kết quả đạt được

Hệ thống đã đạt các mục tiêu chính:

- Có prototype web hoạt động với giao diện dashboard, upload và viewer.
- Có đăng nhập bằng MetaMask.
- Có mã hóa phía client và lưu ciphertext.
- Có token truy cập, thu hồi token và hủy tài liệu.
- Có luồng phê duyệt A+B cho C.
- Có audit trail và kiểm thử tự động.
- Có cấu hình local/R2/Railway phục vụ demo.

5.6.2. Hạn chế

Một số hạn chế cần nêu rõ:

- Protected viewer không thể ngăn người dùng chụp ảnh bằng thiết bị khác.
- SQLite phù hợp prototype một instance, chưa phù hợp scale nhiều instance.
- Chưa có hệ thống notification để báo người duyệt khi có yêu cầu mới.
- Chưa có watermark cá nhân hóa theo người nhận.
- Chưa có audit receipt dạng Merkle tree hoặc bằng chứng zero-knowledge.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Báo cáo đã trình bày quá trình xây dựng WEB3demo theo góc nhìn Kỹ thuật phần mềm. Từ bài toán chia sẻ tài liệu nhạy cảm, nhóm đã phân tích yêu cầu, xác định tác nhân, thiết kế use case, thiết kế kiến trúc, cơ sở dữ liệu, API, giao diện, sau đó cài đặt và kiểm thử prototype.

Kết quả quan trọng nhất của đề tài là chứng minh được một luồng bảo vệ tài liệu có tính hệ thống: tài liệu được mã hóa trước khi upload, quyền truy cập được cấp bằng token, tài liệu nhạy cảm có thể yêu cầu nhiều người phê duyệt, và chủ sở hữu có thể thu hồi token hoặc hủy tài liệu. Hệ thống cũng có dashboard, viewer, audit trail và cấu hình triển khai lên Railway/R2.

Đối với môn Kỹ thuật phần mềm, đề tài thể hiện đầy đủ các khía cạnh của một quy trình phát triển phần mềm: yêu cầu, thiết kế, cài đặt, kiểm thử, triển khai và đánh giá. Đây không chỉ là một chức năng mã hóa/giải mã, mà là một prototype có nhiều module phối hợp với nhau.

6.2. Bài học rút ra

Trong quá trình thực hiện, nhóm rút ra một số bài học:

- Cần phân biệt rõ giữa bảo mật bằng mã hóa và kiểm soát hành vi sau khi người dùng đã xem tài liệu.
- Thiết kế module rõ ràng giúp hệ thống dễ mở rộng từ local storage sang R2.
- Kiểm thử E2E rất cần thiết vì nhiều lỗi chỉ xuất hiện khi các module UI, API, database và session chạy cùng nhau.
- Deploy online không chỉ là đẩy code, mà còn phụ thuộc biến môi trường, secret, volume, healthcheck và quyền ghi runtime.
- Với hệ thống liên quan quyền truy cập, audit log và thông báo lỗi rõ ràng quan trọng không kém chức năng chính.

6.3. Hướng phát triển

Các hướng phát triển ngắn hạn:

- Thêm watermark cá nhân hóa trong protected viewer để giảm rủi ro chia sẻ trái phép.
- Hỗ trợ viewer tốt hơn cho PDF, notebook, video và file khoa học.
- Thêm notification khi có yêu cầu phê duyệt mới hoặc file sắp hết hạn.
- Cải thiện dashboard audit để chủ sở hữu dễ hiểu hành vi truy cập.
- Chuyển SQLite sang PostgreSQL nếu cần nhiều instance hoặc nhiều người dùng.

Các hướng nghiên cứu nâng cao:

- **Merkle audit receipt**: tạo biên nhận audit bất biến sau mỗi phiên chia sẻ.
- **Zero-knowledge permission proof**: chứng minh người dùng có quyền mà server không cần biết toàn bộ thông tin định danh.
- **Time-locked encryption**: khóa chỉ có thể giải mã trong khoảng thời gian nhất định.
- **Risk scoring**: đánh giá rủi ro theo thiết bị, tần suất mở file và hành vi bất thường.

6.4. Luận điểm bảo vệ

Khi bảo vệ, nhóm cần nhấn mạnh rằng hệ thống không tuyên bố làm cho việc copy trở nên bất khả thi. Nếu người dùng đã được xem nội dung, vẫn tồn tại rủi ro chụp màn hình hoặc copy thủ công. Giá trị của hệ thống nằm ở việc làm cho quyền truy cập trở nên có chủ đích, có thể thu hồi, có thể kiểm chứng và khó bị lạm dụng hơn so với gửi file gốc trực tiếp.

TÀI LIỆU THAM KHẢO

1. Nhóm 11, *Mã nguồn hệ thống WEB3demo*, GitHub repository. Truy cập ngày 16/06/2026 tại <https://github.com/Poicitaco/WEB3demo>.
2. Vercel, *Next.js Documentation: App Router and Route Handlers*. Truy cập ngày 16/06/2026 tại <https://nextjs.org/docs>.
3. Meta Platforms, *React Reference*. Truy cập ngày 16/06/2026 tại <https://react.dev/reference/react>.
4. Mozilla Developer Network, *Web Crypto API*. Truy cập ngày 16/06/2026 tại https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API.
5. MetaMask, *MetaMask Developer Documentation*. Truy cập ngày 16/06/2026 tại <https://docs.metamask.io>.
6. Ethers.js, *Ethers Documentation, Signing Messages*. Truy cập ngày 16/06/2026 tại <https://docs.ethers.org/v6>.
7. Cloudflare, *Cloudflare R2 Object Storage Documentation*. Truy cập ngày 16/06/2026 tại <https://developers.cloudflare.com/r2>.
8. Railway, *Railway Documentation: Deployments and Volumes*. Truy cập ngày 16/06/2026 tại <https://docs.railway.com>.
9. SQLite Consortium, *SQLite Documentation*. Truy cập ngày 16/06/2026 tại <https://www.sqlite.org/docs.html>.
10. Microsoft, *Playwright Documentation*. Truy cập ngày 16/06/2026 tại <https://playwright.dev/docs/intro>.
11. OWASP Foundation, *Web Security Testing Guide*. Truy cập ngày 16/06/2026 tại <https://owasp.org/www-project-web-security-testing-guide>.
12. NIST, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM)*, NIST SP 800-38D, 2007. <https://csrc.nist.gov/publications/detail/sp/800-38d/final>.
13. NIST, *Advanced Encryption Standard (AES)*, FIPS PUB 197, 2023. <https://csrc.nist.gov/pubs/fips/197/final>.